

AULA 3

S.O.L.I.D.

Cinco princípios para reduzir acoplamento e tornar uma API Java mais fácil de alterar, testar e entender.

S

RESPONSABILIDADE ÚNICA

O

ABERTO E FECHADO

L

SUBSTITUIÇÃO DE LISKOV

I

SEGREGAÇÃO DE INTERFACES

D

INVERSÃO DE DEPENDÊNCIA

S

OBJETIVO / PAYMENT-API

■ SCOPE DEFINED

Objetivo da aula

Sentir o problema de um código rígido e refatorá-lo sem criar arquitetura desnecessária.

Vamos comparar uma Payment API com muitos `if`, responsabilidades misturadas e dependências concretas com uma versão baseada em classes menores, Strategy e injeção pelo construtor.

02 / 20

ANTERIOR

02 / 20

PRÓXIMO

/ 20

O código funciona. Depois muda.

PIX

Gera chave e QR Code.

Cartão

Mascara número e calcula taxa.

Boleto

Gera número e vencimento.

Próximo tipo

Abre a mesma classe outra vez.

Quando toda mudança exige editar o mesmo service, o risco de regressão cresce junto com o sistema.

LegacyPaymentService

```
public Payment createPayment(Request request) {  
    validate(request);  
    processByType(request);  
    BigDecimal fee = calculateFee(request);  
    Payment payment = buildPayment(request, fee);  
    Payment saved = repository.save(payment);  
    sendEmail(saved);  
    return saved;  
}
```

Validação, tipo de pagamento, taxa, persistência e notificação mudam por motivos diferentes, mas estão presos na mesma classe.

S

VISÃO GERAL / CINCO PERGUNTAS

REVIEW READY

SOLID faz cinco perguntas

S

ESTA CLASSE FAZ COISAS DEMAIS?

O

PRECISO EDITAR CÓDIGO ANTIGO?

L

POSSO TROCAR A IMPLEMENTAÇÃO?

I

O CONTRATO OBRIGA DEMAIS?

D

DEPENDO DE DETALHE CONCRETO?

S: uma razão para mudar

Controller

Recebe HTTP, valida a entrada e delega.

Service

Orquestra a regra do caso de uso.

Repository

Conversa com o MongoDB.

Analogia da cozinha



O garçom recebe o pedido, o cozinheiro prepara e o caixa cobra. Se uma pessoa fizer tudo, qualquer mudança vira confusão. Na API, controller, service e repository também precisam de papéis claros.

Responsabilidade única não significa “uma classe com um método”. Significa agrupar aquilo que muda pelo mesmo motivo.

O service passa a orquestrar

```
public Payment createPayment(CreatePaymentRequest request) {  
    PaymentProcessor processor =  
        processorResolver.resolve(request.type());  
  
    ProcessedPayment processed = processor.process(request);  
    Payment payment = PaymentMapper.mapFrom(request, processed);  
  
    return paymentRepository.save(payment);  
}
```

O `PaymentService` não precisa saber gerar QR Code, mascarar cartão ou montar boleto. Ele coordena quem sabe fazer cada parte.

O: extensão sem cirurgia

Fechado para extensão

Cada novo pagamento adiciona outro `else if` no mesmo service.

Aberto para extensão

Um novo tipo entra como uma nova classe que cumpre o contrato.

Analogia da tomada



Para ligar um aparelho novo, você não quebra a parede e refaz a instalação elétrica. O aparelho apenas respeita o padrão da tomada. Uma nova estratégia de pagamento deve respeitar o contrato sem exigir reforma no service.

Adicionar comportamento deve exigir mais código novo e menos alteração em código estável.

Strategy cria o ponto de extensão

```
public interface PaymentProcessor {  
    PaymentType getType();  
    ProcessedPayment process(CreatePaymentRequest request);  
}
```

PixPaymentProcessor

Processa somente PIX.

CardPaymentProcessor

Processa somente cartão.

BoletoPaymentProcessor

Processa somente boleto.

L: trocar sem surpresa

Se o código espera um ``PaymentProcessor``, qualquer implementação válida precisa funcionar naquele lugar.

PIX

Recebe request e devolve resultado processado.

Cartão

Respeita o mesmo contrato e não muda a expectativa.

Boleto

Pode substituir as outras estratégias no fluxo.



Analogia da lâmpada

Se duas lâmpadas usam o mesmo encaixe, você espera trocar uma pela outra e continuar tendo luz. Se uma delas desliga a casa inteira, ela cabe fisicamente, mas quebra o comportamento esperado.

Não prometa o que não suporta

Problema

Uma implementação lança `UnsupportedOperationException` para um método obrigatório da interface.

Melhoria

O contrato contém somente operações que todas as implementações conseguem cumprir.

Liskov não é apenas “usar herança”. É preservar o comportamento esperado por quem depende do contrato.

I: contratos menores

```
public interface PaymentOperations {  
    process();  
    notifyCustomer();  
    exportPdf();  
    refund();  
    generateBoleto();  
}
```

Uma classe de PIX não deveria implementar exportação ou geração de boleto só porque alguém agrupou tudo em uma interface genérica.

Separe por capacidade

PaymentProcessor

Processa um tipo de pagamento.

PaymentNotifier

Notifica o cliente quando necessário.

ReportExporter

Exporta relatórios quando necessário.



Analogia do controle remoto

Um controle de ventilador precisa ligar, desligar e ajustar velocidade. Obrigar esse controle a ter botões de Netflix, volume e troca de canal cria funções que ele nunca poderá cumprir.

Uma classe implementa apenas o contrato que realmente usa. Isso reduz métodos vazios, exceções artificiais e dependências desnecessárias.

D: dependa do contrato

```
public PaymentService() {  
    this.repository =  
        new MongoPaymentRepository();  
}
```

```
public PaymentService(  
    PaymentRepository repository  
) {  
    this.repository = repository;  
}
```

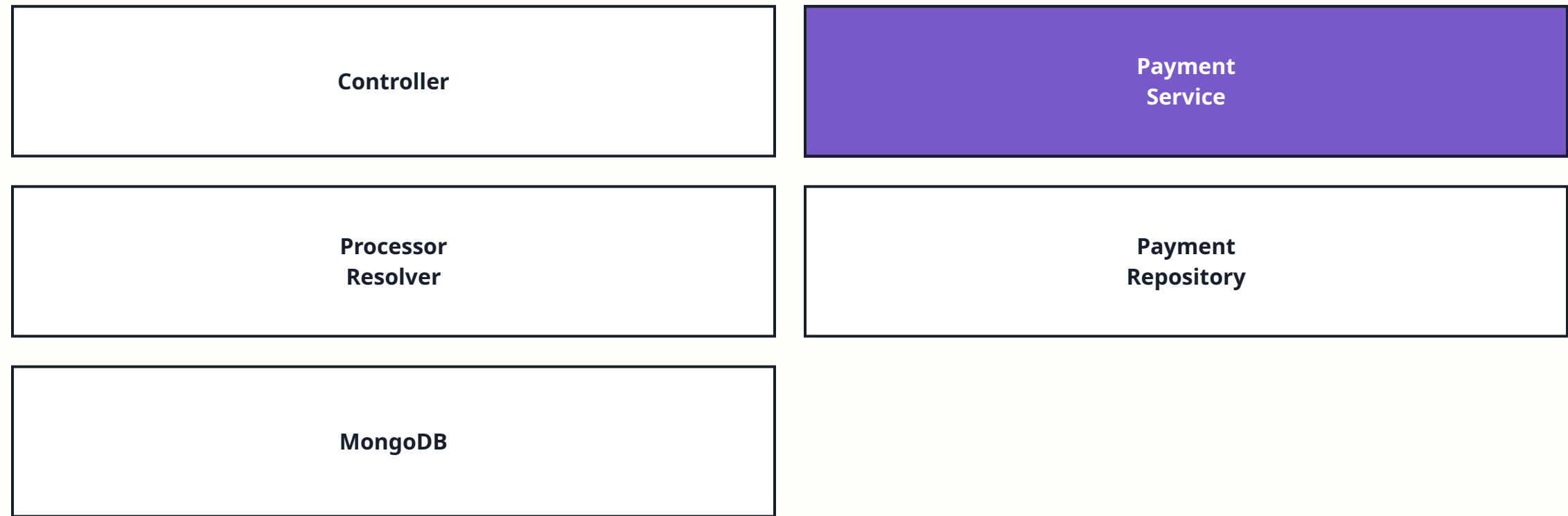


Analogia do carregador

O celular depende de uma entrada USB-C, não da fábrica específica do carregador. Você pode trocar o carregador desde que ele respeite o padrão. O service depende de `PaymentRepository`, não de uma construção concreta feita com `new`.

Criar a dependência com `new` prende a regra ao detalhe concreto.
Receber a dependência pelo construtor deixa o Spring montar o objeto.

O Spring conecta as peças



A regra conhece contratos. O container encontra implementações registradas e injeta tudo pelo construtor.

O que mudou de verdade?

Antes

Uma classe grande, `if` por tipo, detalhes concretos e testes difíceis.

Depois

Service orquestrador, estratégias focadas, contratos pequenos e dependências injetadas.

A melhoria não está em ter mais arquivos. Está em conseguir mudar uma parte sem precisar entender e arriscar todas as outras.

No tutorial prático

Legacy API

Criaremos a versão que mistura responsabilidades de propósito.

Refatoração SOLID

Aplicaremos processors, resolver, mapper e injeção.

Infraestrutura

Spring Boot, MongoDB, Docker e Swagger.

```
POST /api/v1/legacy-payments
POST /api/v1/payments
GET  /api/v1/payments/{id}
```

Erros comuns

- 01

Criar interface sem existir uma necessidade real de variação.
- 02

Transformar cada método em uma classe e fragmentar demais o código.
- 03

Achar que SOLID significa usar herança em todo lugar.
- 04

Criar uma interface gigante com todas as operações do sistema.
- 05

Colocar regra de negócio no controller, DTO ou model do MongoDB.

S

RESUMO / CINCO DECISÕES

COMPLETE

Resumo

S

SEPARE RAZÕES PARA MUDAR

O

ESTENDA SEM REABRIR TUDO

L

PRESERVE O CONTRATO

I

PREFIRA INTERFACES FOCADAS

D

INJETE ABSTRAÇÕES

SOLID não elimina mudanças. Ele diminui quantas partes precisam ser tocadas quando a mudança chega.

ANTERIOR

20 / 20

INÍCIO